

COMP0008: Computer Architecture and Concurrency

Introduction to the Second Part of the Module

Stefano Vissicchio

Coming up next (in this module)

- High-level topic: **opportunities and perils of modern hardware** from the programmers' perspective
- Some of the questions we will cover:
 - How can we ensure good **hardware performance**?
 - What is the **impact of hardware optimization** on software?
 - How to program concurrent **software that takes advantage** of hardware optimizations?
 - What are the **risks and common bugs of concurrent software**, and how to avoid them?

Coming up next (in this module)

- High-level topic: **opportunities and perils of modern hardware** from the programmers' perspective

Week	Topic	Abstraction level
6	Executing MIPS code in single-core processor	Concurrency in hardware

Coming up next (in this module)

- High-level topic: **opportunities and perils of modern hardware** from the programmers' perspective

Week	Topic	Abstraction level
6	Executing MIPS code in single-core processor	Concurrency in hardware
7	Multi-core processors, threads and concurrency abstraction	Concurrency in software

Coming up next (in this module)

- High-level topic: **opportunities and perils of modern hardware** from the programmers' perspective

Week	Topic	Abstraction level
6	Executing MIPS code in single-core processor	Concurrency in hardware
7	Multi-core processors, threads and concurrency abstraction	Concurrency in software
8	Threads and thread synchronization in Java	Concurrency in Java
9	Java memory model, monitors and conditional synchronization	

Coming up next (in this module)

- High-level topic: **opportunities and perils of modern hardware** from the programmers' perspective

Week	Topic	Abstraction level
6	Executing MIPS code in single-core processor	Concurrency in hardware
7	Multi-core processors, threads and concurrency abstraction	Concurrency in software
8	Threads and thread synchronization in Java	Concurrency in Java
9	Java memory model, monitors and conditional synchronization	
10	Reasoning on correctness of multi-threaded systems	Concurrency in larger applications

How we will cover those topics

- Textbooks and **readings**
 - These are the **main references** for the technical content of the module
 - Complemented by **additional material**
- **Office hours** and **Moodle forum**
 - Optional 1:1 sessions and offline Q&A, for specific questions you may have
- Live **sessions** (pls book 2h slot)
 - I plan to mainly **review core concepts**, and offer my perspective on them
 - If you'd like me to cover **specific topics**, ask in advance via Moodle forum
 - Please do not hesitate to **ask questions** or let me know if I'm too fast/slow!



deeper
and more
complete

higher-level
and broader

How we will cover those topics: readings

- Before each week, I will specify **essential readings** for the content covered in that week
 - Essential means that they are **key to understand core material** for the module
 - You are **required** to read them **before office hours**
 - You may also want to check them out *before live sessions*
 - Essential does **not** mean **exclusive!**
 - You are **encouraged to read more**, especially if you need for understanding concepts or are interested in more details
- I will additionally post **further readings** on Moodle
 - They are suggestions to **complement** essential readings

How we will cover those topics: additional

- Slides from in-person sessions
 - NB: would you like (some) slides to be uploaded on Moodle before sessions?

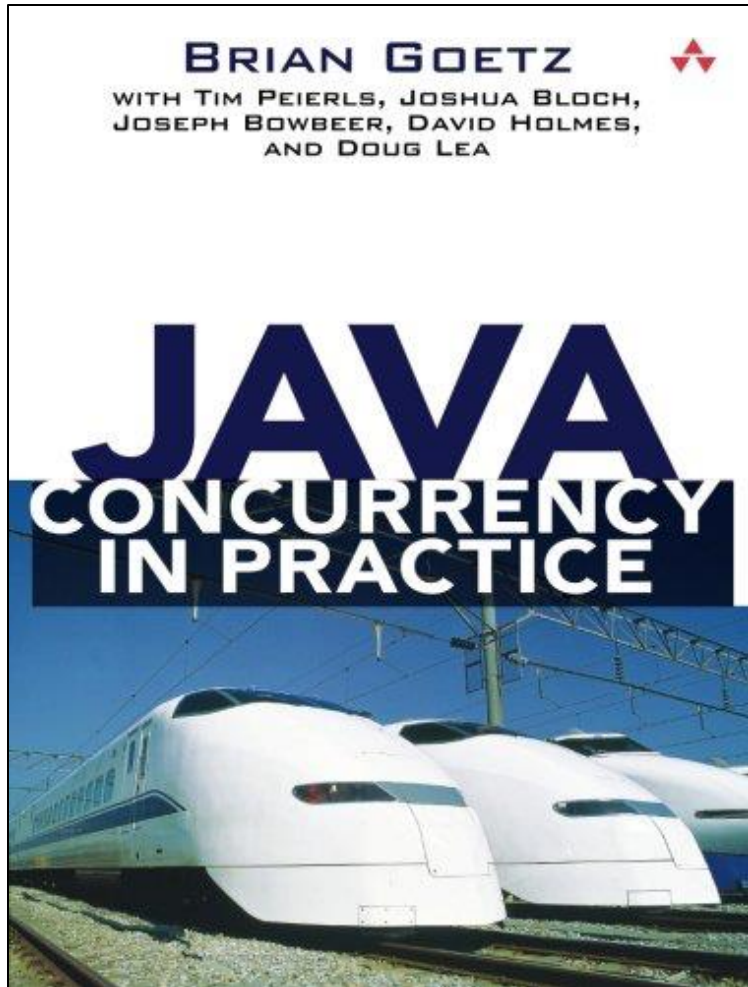
How we will cover those topics: additional

- **Slides** from in-person sessions
 - NB: would you like (some) slides to be uploaded on Moodle before sessions?
- **Additional material**: more slides, code and references
 - to better understand concepts, and expand on topics
- **Non-assessed courseworks**: exercises useful to test your understanding and prepare for the exam
 - you are suggested to attempt CWs on your own **before solutions are discussed in live sessions**

How we will cover those topics: notes

- Content of essential readings, in-person sessions and additional material is **all examinable**
 - However, the focus is on the **concepts**, not on specific sentences or details
 - We will examine your ability to **apply** the presented concepts, and **build upon** them
 - Note: further readings are *not* examinable
- **Office hours** (proposal)
 - Just after each live session, in person
 - i.e., 1-2 pm @169 Euston Road, room 206 (or outside classroom)

Main textbook



- Referred to as [GPBBHL] on Moodle and slides
- Available through UCL library service
 - paper copies are also sold on Amazon
- Magee-Kramer's book is optional
 - may be useful for more code examples than in additional material

Intended learning outcomes

- Primary goals
 - cover the **fundamentals** of software concurrency
 - provide you with **future-proof knowledge**
- A few observations on the module content & delivery
 - The textbook refers to a version of Java which is not so recent, **but** it covers very well the **core concurrency problems and solutions** of concurrency in Java (and other OO languages)
 - All primitives, patterns, idioms and code examples are in Java, **but** the main **underpinning principles and lessons learned** have broader validity
 - We'll mostly focus on basic concurrency primitives (e.g., locks), **but** understanding them will enable you to **easily learn about other constructs and libraries** in the future